PROJECT-BASED LEARNING

PHASE-II EVALUATION

Maze Lab: Interactive Maze Generation and Comparative Pathfinding

Team ID: T272 | Team Blockbuster

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: T272 (Team Blockbuster)
Semester: 3rd Semester
Course Code: DSCPP-III-2026-T272
Domain: Graph Algorithms & OOP

Team Members

1. Tanmay Kumar (Lead) – 2510370439
2. Sparsh Saxena – 2510370313
3. Hemang Sethi – 2510370341

Mentor

Mr. Sachin Bhatt

Interactions completed: 4 Sessions

Brief Discussion: Team details, project domain, mentor guidance, and individual task ownership.

Phase-I Recap & Feedback Incorporated

Phase-I Status

Successfully designed modular strategy interfaces (`MazeGenerator`, `MazeSolver`), implemented base `Cell` and `Maze` classes, and completed terminal rendering.

Feedback Received

- Prevent potential stack overflow in large DFS mazes.
- Ensure shared wall mutations stay synchronized.
- Standardize solver metrics for fair comparative benchmarks.

Changes Incorporated

- Converted DFS generator to explicit heap stack.
- Centralized wall modifications through `Maze`.
- Standardized metrics payload across `MazeSolver`.

Brief Discussion: Summarized Phase-I deliverables and integrated structural architectural feedback.

Problem Statement & Finalized Objectives

Problem Statement

Bridging abstract theoretical Data Structures & Algorithms (graphs, stacks, priority queues) with practical Object-Oriented C++ design within a unified interactive simulation.

Finalized Objectives

1. Generate perfect 2D grid mazes using iterative DFS and Kruskal's algorithm.
2. Support player manual navigation with boundary wall collision detection.
3. Perform pathfinding solving using BFS and A* (Manhattan heuristic).
4. Record execution time, path length, and visited node metrics for algorithm benchmarking.

Brief Discussion: Core focus remains on interactive evaluation between human navigation and graph pathfinders.

Updated System Architecture / Workflow



Major Components

- `Maze` & `Cell`: Grid topology & walls.
- `MazeGenerator`: Abstract generation strategy.
- `MazeSolver`: Abstract pathfinding strategy.

Changes Since Phase-I

- Centralized wall mutation interface.
- Explicit stack iteration for DFS generation.
- Unified benchmarking metric payload.

Brief Discussion: System architecture uses the Strategy Pattern to decouple generation and pathfinding algorithms.

Module Design & Functional Requirements

Major Modules

1. Grid Engine (`Cell`, `Maze`)
2. Generators (`DFSGenerator`, `Kruskal`)
3. Interactive Controls (`Player`)
4. Solvers (`BFSSolver`, `AStarSolver`)

Functional Requirements

- Guarantee grid connectivity without closed loops.
- Validate movement boundaries against cell walls.
- Construct shortest paths via BFS parent maps.
- Benchmarking steps, evaluated cells, & execution time.

Module Dependency

Generators and Solvers depend strictly on `Maze` via abstract strategy interfaces without internal coupling.

Brief Discussion: Explains module separation and interface contracts.

Technology Stack & Development Methodology

Technology Stack

- **Language:** C++ (ISO C++17 Standard)
- **Data Structures:** `std::vector`, `std::priority_queue`, `std::stack`, Custom DSU
- **Build Tool:** g++ / CMake
- **Version Control:** Git & GitHub
- **Optional GUI:** SDL2 (Stretch Goal)

Development Methodology

1. Interface Contracts & Core Data Models
2. Iterative Generation & Traversal Solvers
3. Terminal Display & Interactive Controls
4. Metrics Engine & Benchmark Tests
5. Validation & Documentation

Brief Discussion: Selected C++17 for performance, low memory footprint, and object-oriented abstraction.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C / C++	Explicit Stacks, Queue Traversals, Priority Queues, Disjoint Set Union (DSU), 2D Array Grids	`DFSGenerator`, `BFSSolver`, `AStarSolver`, `DisjointSet`
OOPs with C++	Abstract Interfaces, Strategy Pattern, Class Encapsulation, Polymorphism, RAII	`MazeGenerator`, `MazeSolver`, `Maze`, `Player`

Important: Integrates C++ OOP abstractions directly with foundational C/C++ graph algorithms in a single system.

Brief Discussion: Technical mapping of course concepts to operational codebase modules.

Development & Implementation Progress

Module	Status	Work Completed
Grid & Cell Models	Completed	2D grid representation, wall operations, boundary check methods
DFS Generator	Completed	Iterative stack-based backtracking maze generator
BFS Solver	Completed	Breadth-first search with parent tracking & path reconstruction
Player Movement	Completed	Keypress controls with collision rejection against active walls
A* Pathfinder	In Progress	Priority queue configuration with Manhattan distance heuristic
Kruskal Generator	In Progress	Shared wall list setup with Disjoint Set Union (DSU) cycle prevention

Overall Project Completion: 65% Completed

Brief Discussion: Core interactive loop and primary graph algorithms are functional.

Working Prototype / Demonstration

Implemented Features

- Dynamic maze grid generation (Randomized DFS).
- Real-time player navigation in terminal.
- Collision checks rejecting movement into walls.
- Automated BFS path reconstruction.

Terminal Interface Output

```
+---+---+---+---+---+
| P | | | |   (P = Player)
+ + +---+ + +
| | | G | | |   (G = Goal)
+---+ + + +---+
| | |   (* = BFS Path)
+---+---+---+---+---+
```

Status: Core generate → play → solve loop fully operational.

Brief Discussion: Terminal UI displays generated maze, player position, and path solution.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Maze Connectivity (DFS)	All cells accessible from start position	Zero isolated cells detected across grids	Pass
Wall Collision (Player)	Invalid moves into walls rejected	Player position remains unchanged	Pass
BFS Path Validity	Continuous valid path ending at goal	Path matches optimal grid traversal	Pass
A* & Kruskal Tests	A* path optimality & Kruskal span	Implementation phase	Pending
Comparative Benchmarking	Fair performance comparison	Scheduled post metrics implementation	Pending

Results / Validation

Structural validation tests pass; algorithm comparative evaluation planned upon completing A* and Kruskal modules.

Brief Discussion: Validation suite verifies grid connectivity and player movement logic.

GitHub Repository & Development Activity

Repository Information

- Repository Branch: `master`
- README.md: Architecture & Build Setup Included
- Contributors: All Team Members Added
- Language: C++17

Important Commits

- `b09bbff` Maze Lab PBL (initial commit)
- `e7b7868` Added C++ source code file
- `c9239f6` Added README.md file
- `9cea4fb` Added main file for Phase 2

Requirement: Individual commits maintained across all members following modular task distribution.

Brief Discussion: Source code, README, reports and presentations are version-controlled in one repository.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Tanmay Kumar	Team Lead	Grid architecture, abstract strategy interfaces, execution flow loop, statistics engine	[XX]
Sparsh Saxena	Core Developer	Iterative DFS generator, BFS solver with parent maps, A* pathfinder implementation	[XX]
Hemang Sethi	UI / Developer	Player input mechanics, collision checking, terminal UI rendering, DSU class	[XX]

Individual Evaluation

Every team member has engineered and verified their assigned codebase modules independently.

Brief Discussion: Technical contributions map directly to individual commits and modules.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Shared wall mutation inconsistencies.
- Risk of call-stack overflow in DFS.
- Representing cell-based walls in Kruskal.

Solutions Adopted

- Funneled wall changes via `Maze` methods.
- Switched to explicit heap-allocated stack.
- Filtered edge creation to right/bottom cells.

Phase-III Roadmap

1. Complete A* solver and Kruskal DSU module.
2. Build metrics engine & comparison menu.
3. Run comprehensive test suite.
4. Final presentation & SDL2 GUI (optional).

Brief Discussion: Addresses current technical roadblocks and outlines clear steps toward final project completion.

Expected Outcomes & References

Expected Final Outcomes

- Fully functional C++ console application.
- Side-by-side performance benchmarking.
- Plug-and-play strategy architecture.
- Interactive human vs algorithm comparison.

References

1. Cormen, T. H., et al. *Introduction to Algorithms*. MIT Press.
2. Gamma, E., et al. *Design Patterns*. Addison-Wesley.
3. ISO/IEC C++17 Standard Specifications.

Thank You
Questions & Suggestions

Brief Discussion: Summary of final deliverables and core academic resources used.